

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Fundada en 1551

FACULTAD DE INGENIERÍA DE SISTEMA E INFORMÁTICA
ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE - LIMA



Proyecto Grupal

Integrantes:

Guevara Chavez Luis Rodrigo

Núñez Cardenas Ivan Joaquin

Bejar Mallma Harian Aaron

Arancivia Salas Christian Gabriel

Rojas Rojas Max Fernando

Docente:

Juan Gamarra Moreno

Asignatura:

Inteligencia Artificial

Bloque A — Programas en Java.....	3
Informe 2 — Juegos con Poda Alfa-Beta (Java).....	3
1. Definición Formal del Problema.....	3
2. Modelado formal del juego.....	3
3. Diseño de la solución - Algoritmos implementados.....	5
Justificación teórica.....	12
4. Funcionalidades del programa.....	13
5. Casos de prueba y resultados experimentales.....	13
Escenario 1 — Partida completa Humano vs Máquina.....	13
Escenario 3 — Comparación por profundidad de búsqueda.....	18
Escenario 4 — Poda desde una posición específica.....	19
6. Análisis de resultados y discusión.....	20
7. Conclusiones.....	21
8. Documentación de prompts.....	21
9. Referencias.....	26

Bloque A — Programas en Java

Informe 2 — Juegos con Poda Alfa-Beta (Java)

Objetivo: implementar un agente inteligente para un juego adversarial de dos jugadores mediante Minimax con poda Alfa-Beta.

Problema utilizado: TicTacToe (3 en raya)

1. Definición Formal del Problema

El Tic-Tac-Toe (o 3 en raya) es uno de los juegos de mesa más simples desde el punto de vista combinatorio, pero constituye un excelente conjunto de pruebas para introducir los fundamentos de los algoritmos MinMax en Inteligencia Artificial. Al tratarse de un juego determinista, de suma cero, de información perfecta y con un espacio de estados finito y pequeño (a lo sumo $9! = 362\,880$ secuencias de jugadas), permite implementar y validar de forma completa algoritmos que en juegos más complejos sólo pueden aplicarse de forma parcial o aproximada.

2. Modelado formal del juego

Siguiendo el enfoque clásico de formulación de problemas de búsqueda en juegos (Russell & Norvig), el Tic-Tac-Toe se define formalmente mediante la tupla (S, s_0 , JUGADOR, ACCIONES, RESULTADO, TERMINAL, UTILIDAD):

Estados (S)

Un estado se representa como una matriz de caracteres de 3×3, donde cada celda toma uno de tres valores: 'X' (ficha de la Máquina), 'O' (ficha del Humano) o '.' (celda vacía). En el código, esta representación corresponde a la clase Board, que encapsula el arreglo `char[3][3]` y las operaciones sobre él.

Estado inicial (s_0)

El estado inicial es el tablero completamente vacío, es decir, las 9 celdas contienen el carácter '.'.

Jugadores

- MAX: la máquina, juega con la ficha X. Busca maximizar la utilidad del estado terminal.
- MIN: el humano, juega con la ficha O. Busca minimizar la utilidad del estado terminal.

La función JUGADOR(s) alterna entre MAX y MIN en cada turno, y al principio de la partida se elige quien es el que empieza el juego.

Jugadas legales (ACCIONES)

ACCIONES devuelve el conjunto de celdas vacías del tablero s. Cada acción se representa como un par (fila, columna) con valores que están entre (0, 1, 2). La función RESULTADO(s, a) devuelve el estado resultante de colocar la ficha del jugador en turno en la celda indicada por a.

Función terminal - Estados meta

Un estado s es terminal si se cumple alguna de las siguientes condiciones:

- Victoria de MAX: existen 3 fichas 'X' alineadas (fila, columna o diagonal) → UTILIDAD(s) = +10.
- Victoria de MIN: existen 3 fichas 'O' alineadas → UTILIDAD(s) = -10.
- Empate: el tablero está lleno y no hay 3 en raya → UTILIDAD(s) = 0.
- No terminal ninguna de las anteriores; el juego continúa.

En la implementación (MinimaxAgent.minimax), la utilidad terminal se ajusta ligeramente con la profundidad para que el agente prefiera victorias más rápidas y derrotas más lentas cuando existan varias líneas de juego con el mismo resultado final, sin alterar el signo de la utilidad ni, por tanto, la decisión óptima entre ganar/perder/empatar.

Función de evaluación heurística

Cuando la búsqueda se limita a una profundidad máxima menor que la necesaria para alcanzar un estado terminal, se recurre a una función de evaluación heurística que aproxima la utilidad de un estado no terminal. Para cada una de las 8 líneas posibles del tablero se determina si la línea está abierta para un jugador, es decir, si no contiene ninguna ficha del oponente. Este valor heurístico se mantiene en un rango acotado entre -8; +8, para no confundirse con los valores terminales (+10 y -10), preservando la coherencia de la comparación entre nodos evaluados de forma heurística y nodos terminales.

3. Diseño de la solución - Algoritmos implementados

Board — modelo del tablero

Representa el estado del juego y las operaciones fundamentales sobre él: colocar/deshacer una jugada, listar jugadas legales, y determinar si hay un ganador o el tablero está lleno. El algoritmo más relevante es checkWinner(), que recorre las 8 líneas posibles del tablero (3 filas, 3 columnas, 2 diagonales) representadas como tripletas de coordenadas.

```
12 public class Board { 70 usages
13
14     public static final char EMPTY = '\u00B7'; // '.' 8 usages
15     public static final char MAX_PLAYER = 'X'; // Máquina (maximiza la utilidad) 29 usages
16     public static final char MIN_PLAYER = 'O'; // Humano u oponente (minimiza la utilidad) 16 usages
17
18     private final char[][] cells; 17 usages
19
20     /** Estado inicial: tablero vacío. */
21     public Board() { 5 usages
22         cells = new char[3][3];
23         for (char[] row : cells) {
24             java.util.Arrays.fill(row, EMPTY);
25         }
26     }
27
28     /** Constructor de copia (para no mutar estados durante la búsqueda). */
29     public Board(Board other) { 5 usages
30         cells = new char[3][3];
31         for (int i = 0; i < 3; i++) {
32             cells[i] = other.cells[i].clone();
33         }
34     }
35
36     > public char get(int row, int col) { return cells[row][col]; }
37
38
39     > public boolean isEmptyCell(int row, int col) { return cells[row][col] == EMPTY; }
40
41
42
43
44     /** Aplica una jugada (transición de estado). */
45     > public void placeMove(int row, int col, char player) { cells[row][col] = player; }
46
47
48
49     /** Deshace una jugada (usado por la búsqueda para no copiar tableros en cada nodo). */
```

```

*/
public char checkWinner() { 4 usages
    int[][] lines = {
        {0,0,0,1,0,2}, {1,0,1,1,1,2}, {2,0,2,1,2,2}, // filas
        {0,0,1,0,2,0}, {0,1,1,1,2,1}, {0,2,1,2,2,2}, // columnas
        {0,0,1,1,2,2}, {0,2,1,1,2,0}                // diagonales
    };
    for (int[] l : lines) {
        char a = cells[l[0]][l[1]];
        char b = cells[l[2]][l[3]];
        char c = cells[l[4]][l[5]];
        if (a != EMPTY && a == b && b == c) {
            return a;
        }
    }
    return EMPTY;
}

/** true si el estado es terminal (victoria de alguno o empate). */
public boolean isTerminal() { return checkWinner() != EMPTY || isFull(); }

/** Todas las líneas del tablero (8 en total), usadas por el evaluador heurístico. */
public static int[][] allLines() { 1 usage
    return new int[][] {
        {0,0,0,1,0,2}, {1,0,1,1,1,2}, {2,0,2,1,2,2},
        {0,0,1,0,2,0}, {0,1,1,1,2,1}, {0,2,1,2,2,2},
        {0,0,1,1,2,2}, {0,2,1,1,2,0}
    };
}

```

Este método se invoca en cada nodo del árbol de búsqueda (dentro de minimax()), por lo que su costo —constante, $O(1)$, al recorrer siempre exactamente 8 líneas fijas— es determinante para el rendimiento global: es, junto con la generación de jugadas legales, la operación más frecuente de todo el programa.

Evaluator — función de evaluación heurística

Implementa la función de evaluación heurística: para cada una de las 8 líneas del tablero, determina si está abierta exclusivamente para MAX, exclusivamente para MIN, o disputada por ambos (caso en que no cuenta para ninguno). El algoritmo es una única pasada sobre las 8 líneas:

```

* Este valor heurístico se mantiene deliberadamente en un rango pequeño
* (típicamente [-8, 8]) para no solaparse con las utilidades terminales
* (+10 / -10 / 0), preservando la consistencia del algoritmo Minimax.
*/
public class Evaluator { 2 usages

    public static int evaluate(Board board) { 1 usage
        int maxLines = 0;
        int minLines = 0;
        char[][] cells = board.getCells();

        for (int[] line : Board.allLines()) {
            char a = cells[line[0]][line[1]];
            char b = cells[line[2]][line[3]];
            char c = cells[line[4]][line[5]];

            boolean hasMax = (a == Board.MAX_PLAYER || b == Board.MAX_PLAYER || c == Board.MAX_PLAYER);
            boolean hasMin = (a == Board.MIN_PLAYER || b == Board.MIN_PLAYER || c == Board.MIN_PLAYER);

            if (hasMax && !hasMin) {
                maxLines++;
            } else if (hasMin && !hasMax) {
                minLines++;
            }
        }
        return maxLines - minLines;
    }
}

```

Minimax sin poda (versión base)

El algoritmo Minimax realiza una búsqueda exhaustiva en profundidad sobre el árbol de juego. El algoritmo se basa en el principio de maximizar la propia utilidad asumiendo que el oponente jugará de manera óptima para minimizar (Russell & Norvig, 2020). En cada nodo correspondiente al turno de MAX, se elige la jugada que maximiza el valor minimax de los estados sucesores; en cada nodo correspondiente al turno de MIN, se elige la jugada que minimiza dicho valor. La recursión termina al llegar a un estado terminal (o al alcanzar la profundidad máxima configurada, en cuyo caso se usa la heurística de evaluación). El método público findBestMove() —invocado una vez por cada jugada de la Máquina— itera sobre las jugadas legales del nodo raíz, evalúa cada una recursivamente y conserva la lista de jugadas que alcanzan el valor óptimo, para finalmente elegir una al azar entre ellas.

Minimax con poda Alfa-Beta (versión optimizada)

La poda Alfa-Beta es una optimización que produce exactamente la misma decisión que el Minimax puro, pero evita explorar ramas del árbol que no pueden influir en el resultado final. Se mantienen dos valores durante el recorrido:

- α (alfa): la mejor utilidad (más alta) garantizada hasta el momento para MAX en el camino hacia la raíz.
- β (beta): la mejor utilidad (más baja) garantizada hasta el momento para MIN en el camino hacia la raíz.

Cuando en un nodo se cumple $\beta \leq \alpha$, se produce un corte (poda): el jugador en el nivel superior del árbol ya tiene una alternativa al menos tan buena como cualquier resultado que pudiera obtenerse continuando la exploración del nodo actual, por lo que explorar el resto de sus sucesores es computacionalmente inútil.

```
private int minimax(Board board, int depth, boolean maximizing, int alpha, int beta, SearchStats stats) {
    stats.nodesEvaluated++;
    stats.maxDepthReached = Math.max(stats.maxDepthReached, depth);

    char winner = board.checkWinner();
    if (winner == Board.MAX_PLAYER) return 10 - depth; // victoria más rápida = mejor
    if (winner == Board.MIN_PLAYER) return depth - 10; // derrota más lenta = menos mala
    if (board.isFull()) return 0; // empate
    if (depth >= maxDepth) return Evaluator.evaluate(board); // corte por profundidad

    if (maximizing) {
        int best = Integer.MIN_VALUE;
        for (int[] move : orderedMoves(board)) {
            board.placeMove(move[0], move[1], Board.MAX_PLAYER);
            int value = minimax(board, depth: depth + 1, maximizing: false, alpha, beta, stats);
            board.undoMove(move[0], move[1]);
            best = Math.max(best, value);
            if (useAlphaBeta) {
                alpha = Math.max(alpha, best);
                if (beta <= alpha) break; // PODA: MIN no permitirá llegar aquí
            }
        }
        return best;
    } else {
        int best = Integer.MAX_VALUE;
        for (int[] move : orderedMoves(board)) {
            board.placeMove(move[0], move[1], Board.MIN_PLAYER);
            int value = minimax(board, depth: depth + 1, maximizing: true, alpha, beta, stats);
            board.undoMove(move[0], move[1]);
            best = Math.min(best, value);
            if (useAlphaBeta) {
                beta = Math.min(beta, value);
                if (beta <= alpha) break; // PODA: MAX no permitirá llegar aquí
            }
        }
        return best;
    }
}
```

Profundidad limitada

El agente admite una profundidad máxima de búsqueda configurable (parámetro `maxDepth`). Por defecto se usa `Integer.MAX_VALUE`, lo que equivale a buscar hasta el estado terminal en cualquier rama (profundidad total, como máximo 9 en Tic-Tac-Toe). Esta flexibilidad permite estudiar el comportamiento del algoritmo y el efecto de la poda en función de la profundidad.

Game — orquestación de partidas

Coordina el flujo de una partida completa en sus dos modalidades, delegando toda decisión de juego a `MinimaxAgent` y toda regla del juego a `Board`. Sus dos algoritmos principales son:

- `playHumanVsMachine(useAlphaBeta, maxDepth, startingPlayer)`: recibe ahora el jugador inicial como parámetro (mejora solicitada), y alterna turnos entre `Board.MAX_PLAYER` y `Board.MIN_PLAYER` a partir de ese valor en lugar de asumir siempre que abre la Máquina.

- playMachineVsMachine(maxDepth, verbose, accumulator): ejecuta una partida completa entre dos agentes, acumulando —si se provee un arreglo accumulator— las métricas comparativas (nodos con/sin poda, tiempos, número de jugadas).

```

9      public class Game { 8 usages
20      *                      Board.MIN_PLAYER si el humano abre el juego.
21      */
22      public void playHumanVsMachine(boolean useAlphaBeta, int maxDepth, char startingPlayer) { 2 usages
23          Board board = new Board();
24          MinimaxAgent machine = new MinimaxAgent(useAlphaBeta, maxDepth);
25          // Agente "espejo" solo para fines estadísticos (no decide jugadas)
26          MinimaxAgent comparisonAgent = new MinimaxAgent(!useAlphaBeta, maxDepth);
27
28          char turn = startingPlayer;
29          System.out.println("=== Humano (O) vs Máquina (X) ===");
30          System.out.println("Algoritmo de la máquina: " + (useAlphaBeta ? "Minimax + Poda Alfa-Beta" : "Minima
31          System.out.println("Empieza: " + (startingPlayer == Board.MAX_PLAYER ? "la Máquina (X)" : "el Humano
32          board.print();
33
34          long totalNodesMachine = 0;
35          long totalNodesComparison = 0;
36          long totalTimeMachine = 0;
37          int moveCount = 0;
38
39          while (!board.isTerminal()) {
40              if (turn == Board.MAX_PLAYER) {
41                  SearchStats stats = new SearchStats();
42                  int[] move = machine.findBestMove(board, Board.MAX_PLAYER, stats);
43                  board.placeMove(move[0], move[1], Board.MAX_PLAYER);
44
45                  SearchStats compStats = new SearchStats();
46                  Board beforeMove = new Board(board);
47                  beforeMove.undoMove(move[0], move[1]);
48                  comparisonAgent.findBestMove(beforeMove, Board.MAX_PLAYER, compStats); // no se usa el result
49
50                  totalNodesMachine += stats.nodesEvaluated;

```



```

/** Máquina vs Máquina; retorna el ganador ('X', 'O' o EMPTY=empate) y acumula stats en el arreglo dado {
public char playMachineVsMachine(int maxDepth, boolean verbose, long[] accumulator) { 3 usages
    Board board = new Board();
    MinimaxAgent agentX = new MinimaxAgent(useAlphaBeta: true, maxDepth); // decide con poda (más rápido)
    MinimaxAgent agentO = new MinimaxAgent(useAlphaBeta: true, maxDepth);
    MinimaxAgent noPodaShadow = new MinimaxAgent(useAlphaBeta: false, maxDepth); // solo estadística

    char turn = Board.MAX_PLAYER;
    if (verbose) {
        System.out.println("== Máquina (X) vs Máquina (O) ==");
        board.print();
    }

    while (!board.isTerminal()) {
        MinimaxAgent agent = (turn == Board.MAX_PLAYER) ? agentX : agentO;
        SearchStats stats = new SearchStats();
        int[] move = agent.findBestMove(board, turn, stats);
        board.placeMove(move[0], move[1], turn);

        if (accumulator != null) {
            SearchStats shadowStats = new SearchStats();
            Board copy = new Board(board);
            copy.undoMove(move[0], move[1]);
            noPodaShadow.findBestMove(copy, turn, shadowStats);
            accumulator[0] += shadowStats.nodesEvaluated; // sin poda
            accumulator[1] += stats.nodesEvaluated; // con poda
            accumulator[2] += shadowStats.elapsedNanos;
            accumulator[3] += stats.elapsedNanos;
            accumulator[4] += 1;
        }
    }
}

```

ExperimentRunner — batería de pruebas de rendimiento

Automatiza los cuatro escenarios de prueba exigidos por el proyecto, reutilizando Game y MinimaxAgent: (1) simulación de N partidas Máquina vs Máquina con acumulación de métricas; (2) comparación de nodos evaluados variando la profundidad máxima (3, 5, 7, 9) desde el estado inicial, ejecutando Minimax puro y Alfa-Beta sobre copias independientes del mismo tablero; y (3) evaluación de la poda desde una posición intermedia fija, construida colocando manualmente cuatro fichas sobre un tablero nuevo.

```

8      public class ExperimentRunner { 2 usages
10          public void runAll() { 2 usages
12              experimentoProfundidades(new int[]{3, 5, 7, 9});
13              experimentoPosicionEspecific();
14          }
15
16          /** Prueba 1: N partidas Máquina vs Máquina, promedio de nodos y tiempos. */
17          public void experimentoPartidasMultiples(int n) { 1 usage
18              System.out.println("\n===== EXPERIMENTO 1: " + n + " partidas Máquina vs Máquina =====");
19              Game game = new Game(scanner, null);
20              long[] acc = new long[5]; // nodosNoPoda, nodosPoda, tiempoNoPoda, tiempoPoda, numJugadas
21              int xWins = 0, oWins = 0, draws = 0;
22
23              for (int i = 0; i < n; i++) {
24                  char winner = game.playMachineVsMachine(Integer.MAX_VALUE, verbose: false, acc);
25                  if (winner == Board.MAX_PLAYER) xWins++;
26                  else if (winner == Board.MIN_PLAYER) oWins++;
27                  else draws++;
28              }
29
30              long totalMoves = acc[4];
31              System.out.println("Resultados: X gana=" + xWins + ", O gana=" + oWins + ", empates=" + draws);
32              System.out.println("Total de jugadas (máquina) evaluadas en las " + n + " partidas: " + totalMoves);
33              System.out.printf("Nodos totales SIN poda: %d (promedio/jugada: %.1f)%n", acc[0], acc[0] / (double) totalMoves);
34              System.out.printf("Nodos totales CON poda: %d (promedio/jugada: %.1f)%n", acc[1], acc[1] / (double) totalMoves);
35              Game.printReduction(acc[0], acc[1]);
36              System.out.printf("Tiempo total SIN poda: %.3f ms | CON poda: %.3f ms%n", acc[2] / 1_000_000.0, acc[3] / 1_000_000.0);
37          }
38
39          /** Prueba 2: comparar nodos evaluados sin/con poda para varias profundidades máximas. */
40          public void experimentoProfundidades(int[] depths) { 1 usage
41              System.out.println("\n===== EXPERIMENTO 2: comparación por profundidad (desde estado inicial) =====");

```

Main — punto de entrada, CLI y menú interactivo

Ofrece dos formas de uso equivalentes: argumentos de línea de comandos (parseArgs, un parser simple de pares --clave valor) y un menú interactivo por consola (interactiveMenu). Ambos caminos convergen en las mismas llamadas a Game y ExperimentRunner. Main también centraliza la configuración de la codificación UTF-8 de la salida estándar (para mostrar correctamente tildes y el carácter '.' en cualquier plataforma) y el manejo seguro de fin de entrada (safeReadLine) en todos los prompts del menú, evitando que el programa termine con una excepción no controlada si la entrada se agota inesperadamente.

```

3 public class Main {
55     private static int parseDepth(String value) { 1 usage
63     }
64
65     @ private static void interactiveMenu(Scanner scanner) { 1 usage
66         System.out.println("=====");
67         System.out.println(" AGENTE INTELIGENTE PARA TIC-TAC-TOE (MINIMAX)");
68         System.out.println(" Curso: Inteligencia Artificial");
69         System.out.println("=====");
70
71         while (true) {
72             System.out.println("\nSeleccione una opción:");
73             System.out.println("1) Jugar Humano (O) vs Máquina (X)");
74             System.out.println("2) Simular Máquina (X) vs Máquina (O)");
75             System.out.println("3) Ejecutar batería de experimentos (100 partidas, profundidades, posición e
76             System.out.println("4) Salir");
77             System.out.print("Opción: ");
78             if (!scanner.hasNextLine()) {
79                 System.out.println("\n[Aviso] Entrada agotada (EOF). Fin del programa.");
80                 return;
81             }
82             String option = safeReadLine(scanner);
83
84             switch (option) {
85                 case "1" -> {
86                     boolean ab = askAlgorithm(scanner);
87                     int depth = askDepth(scanner);
88                     char first = askStartingPlayer(scanner);
89                     new Game(scanner).playHumanVsMachine(ab, depth, first);
90                 }
91                 case "2" -> {
92                     int depth = askDepth(scanner);

```

Justificación teórica

Complejidad de Minimax

Minimax es un algoritmo completo en juegos con un espacio de estados finito, como el Tic-Tac-Toe: dado que el árbol de juego tiene profundidad acotada (máximo 9 jugadas) y factor de ramificación finito (a lo sumo 9), la recursión siempre termina en un número finito de pasos, alcanzando en toda rama un estado terminal. Por lo tanto, el algoritmo siempre encuentra una solución (una jugada óptima) si esta existe, sin riesgo de recursión infinita ni de dejar ramas sin explorar quedando indefinido el valor de la raíz.

Optimalidad de Minimax

Minimax es óptimo frente a un oponente racional cuando la evaluación es perfecta, es decir, cuando la búsqueda alcanza siempre estados terminales (profundidad total) o se usa una función de evaluación que preserva el orden de utilidades reales. En Tic-Tac-Toe, al buscar hasta profundidad total, el algoritmo garantiza el mejor resultado posible: la Máquina nunca pierde frente a cualquier estrategia del oponente, y gana siempre que el oponente cometa un error que abra una línea ganadora.

Corrección de la poda Alfa-Beta

La poda Alfa-Beta es una técnica de optimización, no de aproximación: se demuestra que produce exactamente el mismo valor minimax en la raíz (y, por tanto, la misma jugada

óptima) que el Minimax sin poda, para cualquier orden de exploración de las jugadas. La corrección se apoya en el siguiente argumento: si en un nodo MIN se descubre un sucesor con valor v tal que $v \leq \alpha$ (donde α es el mejor valor ya garantizado para MAX en un ancestro), entonces MAX nunca elegirá llegar a este nodo MIN, porque ya dispone de una alternativa al menos tan buena; por lo tanto, el valor exacto que tomaría el resto de los sucesores del nodo MIN es irrelevante para la decisión final, y pueden descartarse sin afectar el resultado en la raíz. El argumento simétrico aplica para los nodos MAX y el valor β .

Eficiencia de la poda Alfa-Beta

Sea b el factor de ramificación del árbol de juego y d la profundidad de búsqueda. El Minimax sin poda evalúa, en el peor caso, $O(b^d)$ nodos, ya que explora exhaustivamente todo el árbol. La poda Alfa-Beta, en el mejor caso (cuando el orden de exploración coloca primero las mejores jugadas en cada nivel), reduce la complejidad a $O(b^{(d/2)})$, lo que equivale a duplicar la profundidad de búsqueda alcanzable con el mismo presupuesto computacional. En el caso promedio (orden aleatorio de jugadas), la mejora observada empíricamente suele situarse en valores cercanos a $O(b^{(d/2)})$, dependiendo de la calidad de la heurística de ordenamiento utilizada. La reducción del espacio de búsqueda mediante los parámetros alfa-beta no altera el resultado final del análisis, manteniendo la optimalidad del algoritmo original (Knuth & Moore, 1975).

4. Funcionalidades del programa

El programa implementado en Java ofrece las siguientes funcionalidades, accesibles tanto por menú interactivo como por argumentos de línea de comandos:

- Modo Humano vs Máquina: el humano juega con 'O' y la máquina con 'X'.
- Modo Máquina vs Máquina: ambos jugadores usan el agente Minimax/Alfa-Beta; útil para pruebas de rendimiento.
- Selección del algoritmo: Minimax sin poda o Minimax con poda Alfa-Beta.
- Configuración de la profundidad máxima de búsqueda.
- Visualización del tablero tras cada jugada, con coordenadas de referencia.
- Reporte de estadísticas al finalizar la partida: nodos evaluados (con y sin poda), nodos podados, porcentaje de reducción, tiempo de ejecución y ganador.
- Validación robusta de la entrada humana (formato, rango de coordenadas y celdas ocupadas), con manejo de excepciones.

5. Casos de prueba y resultados experimentales

Se ejecutaron los cuatro escenarios de prueba solicitados con la versión del agente. El código correspondiente se encuentra en las clases Game y ExperimentRunner, los valores exactos de nodos evaluados pueden variar ligeramente entre ejecuciones cuando existen múltiples caminos óptimos; los valores aquí reportados corresponden a una ejecución representativa.

Escenario 1 — Partida completa Humano vs Máquina

Se jugó una partida completa contra el agente configurado con poda Alfa-Beta, profundidad total y el humano abriendo el juego. La máquina respondió a la primera jugada humana con (1,1), evaluando 9149 nodos gracias a la poda.

```
Seleccione una opción:
1) Jugar Humano (0) vs Máquina (X)
2) Simular Máquina (X) vs Máquina (0)
3) Ejecutar batería de experimentos (100 partidas, profundidades, posición específica)
4) Salir
Opción: 1
Algoritmo: (1) Minimax sin poda (2) Minimax con poda Alfa-Beta [default 2]: 2
Profundidad máxima de búsqueda (ENTER para búsqueda completa hasta terminal): 9
¿Quién empieza? (1) Máquina (2) Humano (3) Al azar [default 1]: 2
=== Humano (0) vs Máquina (X) ===
Algoritmo de la máquina: Minimax + Poda Alfa-Beta
Empieza: el Humano (0)
```

```
=== Humano (0) vs Máquina (X) ===
Algoritmo de la máquina: Minimax + Poda Alfa-Beta
Empieza: el Humano (0)
```

	0	1	2
0	·		·
---+---+---			
1	·		·
---+---+---			
2	·		·

Ingrese su jugada (fila columna), ej. '1 2': 1 1
Humano juega en (1,1)

	0	1	2
0	·		·
---+---+---			
1	·		0

Humano juega en (1,1)

	0	1	2
0	·		·

1	·		0

2	·		·

Máquina juega en (2,0) | nodos=8853, tiempo=31.056 ms, profundidad=8, poda=SI

	0	1	2
0	·		·

1	·		0

2	X		·

Ingrese su jugada (fila columna), ej. '1 2': 2 2

Humano juega en (2,2)

	0	1	2
0	·		·

1	·		0

2	X		0

Máquina juega en (0,0) | nodos=246, tiempo=0.408 ms, profundidad=6, poda=SI

	0	1	2
0	X		·

1	·		0

2	X		0

Humano juega en (1,0)

	0	1	2			
0	X		·		·	
	---+---+---					
1	0		0		·	
	---+---+---					
2	X		·		0	

Máquina juega en (1,2) | nodos=46, tiempo=0.052 ms, profundidad=4, poda=SI

	0	1	2			
0	X		·		·	
	---+---+---					
1	0		0		X	
	---+---+---					
2	X		·		0	

Humano juega en (0,2)

	0	1	2			
0	X		·		0	
	---+---+---					
1	0		0		X	
	---+---+---					
2	X		·		0	

Máquina juega en (0,1) | nodos=4, tiempo=0.014 ms, profundidad=2, poda=SI

	0	1	2			
0	X		X		0	
	---+---+---					
1	0		0		X	
	---+---+---					
2	X		·		0	

```

    0   1   2
0  X | X | 0
   ---+---+---
1  0 | 0 | X
   ---+---+---
2  X | 0 | 0

>>> Resultado: EMPATE

--- Estadísticas acumuladas de la partida (jugadas de la máquina) ---
Jugadas realizadas por la máquina: 4
Nodos evaluados con el algoritmo usado (Alfa-Beta): 9149
Nodos que hubiera evaluado el algoritmo contrario (Minimax puro): 56490
Nodos podados: 47341
Reducción de nodos por poda Alfa-Beta: 83.80%
Tiempo total de cómputo de la máquina: 31.530 ms
```

Escenario 2 — 100 partidas Máquina vs Máquina

Se simularon 100 partidas completas entre dos instancias del agente (ambas jugando de forma óptima con poda Alfa-Beta), registrando en paralelo, para cada jugada, cuántos nodos habría evaluado un Minimax puro equivalente desde la misma posición (con fines exclusivamente estadísticos, sin influir en la jugada elegida).

```

Seleccione una opción:
1) Jugar Humano (O) vs Máquina (X)
2) Simular Máquina (X) vs Máquina (O)
3) Ejecutar batería de experimentos (100 partidas, profundidades, posición específica)
4) Salir
Opción: 3

===== EXPERIMENTO 1: 100 partidas Máquina vs Máquina =====
Resultados: X gana=0, O gana=0, empates=100
Total de jugadas (máquina) evaluadas en las 100 partidas: 900
Nodos totales SIN poda: 62054920 (promedio/jugada: 68949.9)
Nodos totales CON poda: 2507507 (promedio/jugada: 2786.1)
Nodos podados: 59547413
Reducción de nodos por poda Alfa-Beta: 95.96%
Tiempo total SIN poda: 11457.131 ms | CON poda: 579.690 ms
```

Métrica	Valor
Partidas jugadas	100
Resultados (X / O / empate)	0 / 0 / 100

Jugadas totales analizadas	900
Nodos totales SIN poda	62 054 920
Nodos totales CON poda	2 507 507
Promedio de nodos/jugada SIN poda	68 949.9
Promedio de nodos/jugada CON poda	2 786.1
Reducción de nodos por poda Alfa-Beta	95.96 %
Tiempo total SIN poda	11457.131 ms
Tiempo total CON poda	579.690 ms

El resultado de 100 empates sobre 100 partidas confirma el resultado teórico esperado: cuando ambos jugadores actúan de forma óptima (Minimax con búsqueda completa), el Tic-Tac-Toe siempre termina en empate, incluso con la apertura y los desempates aleatorizados. La reducción media de 95.96% en el número de nodos evaluados ilustra el enorme ahorro computacional que aporta la poda incluso en un juego tan pequeño.

Escenario 3 — Comparación por profundidad de búsqueda

Desde el estado inicial (tablero vacío), se calculó el número de nodos evaluados por el Minimax puro y por Alfa-Beta para profundidades máximas 3, 5, 7 y 9, siempre con el mismo criterio de ordenamiento de jugadas.

```
===== EXPERIMENTO 2: comparación por profundidad (desde estado inicial) =====
Profundidad Nodos sin poda   Nodos con poda   Reducción(%)   Tiempo sin poda   Tiempo con poda
3           585             207              64.62          12.8761           0.5753
5          18729            2027             89.18           7.9364            1.0131
7         221625           10215            95.39          60.5332           3.0208
9        549945           16553            96.99          93.7898           3.5368
```

Profundida d	Nodos sin poda	Nodos con poda	Reducción (%)	Tiempo sin poda (ms)	Tiempo con poda (ms)
3	585	207	64.62	12.8761	0.5753
5	18 729	2027	89.18	7.9364	1.0131
7	221 625	10 215	95.39	60.5332	3.0208
9 (total)	549 945	16 553	96.99	93.7898	3.5368

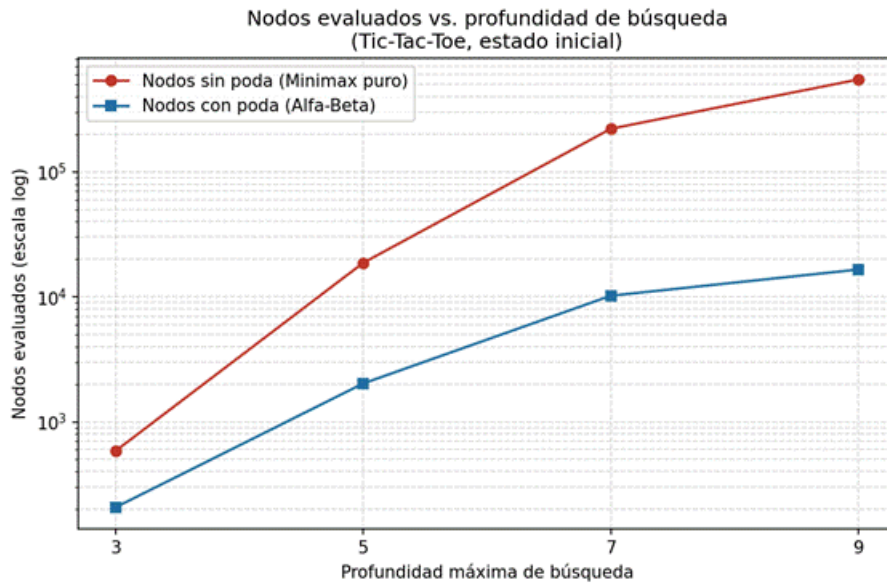


Figura 1. Nodos evaluados (escala logarítmica) en función de la profundidad máxima de búsqueda.

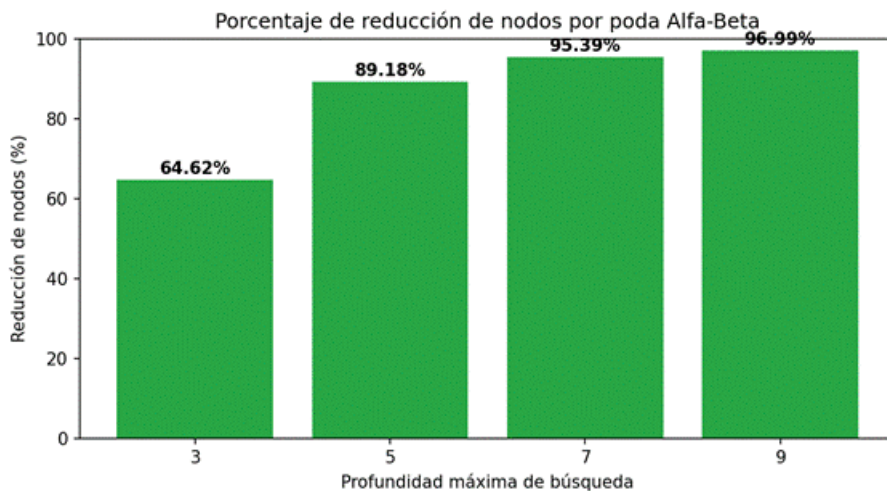


Figura 2. Porcentaje de reducción de nodos evaluados gracias a la poda Alfa-Beta, por profundidad.

Escenario 4 — Poda desde una posición específica

Se evaluó la siguiente posición intermedia, con turno de la Máquina (X):

Ambos algoritmos seleccionan exactamente la misma jugada (2,0) —lo que corrobora la corrección de la poda—, y esta jugada se eligió de forma consistente en 30 de 30 ejecuciones repetidas. Alfa-Beta evaluó un 53.11% menos nodos (94 nodos podados de un total de 177) para llegar a la misma decisión.

```

===== EXPERIMENTO 3: poda desde una posición específica =====
Posición evaluada:

  0   1   2
0  X | . | 0
---+---+---
1  . | 0 | .
---+---+---
2  . | . | X

Turno: Máquina (X)
Jugada elegida (sin poda): (2,0) | nodos=177, tiempo=0.051 ms, profundidad=5, poda=NO
Jugada elegida (con poda): (2,0) | nodos=83, tiempo=0.024 ms, profundidad=5, poda=SI
Nodos podados: 94
Reducción de nodos por poda Alfa-Beta: 53.11%

```

Algoritmo	Jugada elegida	Nodos evaluados	Tiempo (ms)	Profundidad alcanzada
Minimax sin poda	(2,0)	177	0.051	5
Minimax con poda Alfa-Beta	(2,0)	83	0.024	5

6. Análisis de resultados y discusión

Los resultados experimentales muestran de manera consistente que la poda Alfa-Beta reduce drásticamente el número de nodos evaluados sin alterar la calidad de la decisión tomada por el agente, confirmando empíricamente la propiedad teórica de corrección de la poda. Incluso en un árbol de juego tan pequeño como el de TicTacToe (con a lo sumo 549 945 nodos en el peor caso desde el estado inicial), la reducción alcanza el 96.99% a profundidad total, y ya es notable (64.62%) incluso con una profundidad de búsqueda muy limitada (3 niveles).

Esta efectividad se explica por: el árbol de TicTacToe, aunque pequeño en términos absolutos, conserva un factor de ramificación relativamente alto en los primeros niveles, por lo que cualquier corte temprano evita la exploración de subárboles todavía grandes, y la heurística de ordenamiento de movimientos implementada (centro / esquinas / bordes) tiende a presentar primero las jugadas más fuertes, acercando el comportamiento observado al mejor caso teórico $O(b^{(d/2)})$ en lugar del caso promedio.

Cabe notar que las cifras de reducción de esta versión corregida (64.62%–96.99%) son consistentemente menores que las de una versión previa que reutilizaba y estrechaba la ventana α/β entre jugadas hermanas de la raíz (83.76%–98.57%). Se trata de un ejemplo concreto y medible del compromiso entre rendimiento y corrección: la versión más rápida era, en ciertas posiciones, incorrecta.

Es importante notar que en TicTacToe la diferencia en tiempo de ejecución absoluto (ms) resulta poco relevante en la práctica, dado que incluso el Minimax puro completo se ejecuta en menos de 100 ms en hardware moderno. Sin embargo, el valor pedagógico del experimento reside precisamente en que permite observar, en un entorno controlado y verificable, la misma relación de complejidad que hace posible construir agentes competitivos en juegos de mayor escala.

7. Conclusiones

- Se implementó exitosamente un agente inteligente para TicTacToe basado en Minimax, con soporte para poda Alfa-Beta, profundidad de búsqueda configurable y elección de quién abre la partida, usando exclusivamente la biblioteca estándar de Java.
- Se verificó experimentalmente que la poda Alfa-Beta produce siempre la misma jugada que el Minimax sin poda, validando su corrección teórica.
- La poda Alfa-Beta redujo entre el 64.62% y el 96.99% el número de nodos evaluados, resultado consistente con la mejora teórica.
- Se confirmó empíricamente que TicTacToe es un juego resuelto: bajo juego óptimo de ambos jugadores (búsqueda completa con Minimax), el resultado es siempre empate (100 de 100 partidas simuladas), tanto antes como después de introducir la apertura y los desempates aleatorizados.
- El ordenamiento de movimientos (centro, esquinas, bordes) demostró ser un factor relevante para maximizar la efectividad de la poda, acercando el comportamiento observado al mejor caso teórico.

8. Documentación de prompts

N°	P01
Objetivo del prompt	Definir el modelo formal del Tic-Tac-Toe y la estructura base del código en Java, incluyendo representación del tablero, validación de movimientos, verificación de estado terminal y función de utilidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Necesito implementar el juego Tic-Tac-Toe (3 en raya) en Java proporciona el código para 1. La definición formal del juego (estados, jugadores, acciones legales, función terminal, utilidad). 2. Una clase Tablero que: - Represente el tablero 3x3 con caracteres ('X', 'O', ' '). - Valide movimientos (celda vacía). - Verifique si hay ganador (líneas: horizontales, verticales, diagonales). - Verifique si hay empate. - Devuelva utilidad (+10 para X, -10 para O, 0 para empate). 3. Una función de evaluación heurística que calcule líneas

	potenciales para cada jugador. 4. Un método main que permita jugar en consola (humano vs humano) para probar la lógica."
Resultado / cómo se usó	Se obtuvo una clase Board completa con toda la lógica del juego. La función de utilidad y evaluación heurística se implementaron para ser usadas posteriormente por Minimax. El modo humano vs humano permitió validar la corrección de las reglas del juego.
Validación / ajuste del equipo	Se verificaron todas las condiciones de victoria (8 líneas posibles). Se ajustó la evaluación heurística para que ponderara correctamente las líneas abiertas (sin oponente) y líneas bloqueadas. Se añadió un método getMovimientosLegales() que devuelve una lista de posiciones vacías. Se probó con múltiples partidas manuales para asegurar la detección temprana de ganadores.

N°	P02
Objetivo del prompt	Implementar el algoritmo Minimax en su versión completa (sin poda) para Tic-Tac-Toe, con conteo de nodos evaluados y retorno de la mejor jugada.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el Tic-Tac-Toe en Java, necesito implementar el algoritmo Minimax: 1. Clase Minimax que: - Tenga un método findBestMove(Board tablero, char jugador) que retorne la mejor posición (fila, columna). - Tenga un método recursivo minimax(Tablero tablero, int profundidad, char jugador). - Use la función de utilidad del tablero (+10 para MAX, -10 para MIN, 0 para empate). - Cuente el número total de nodos evaluados durante la búsqueda. 2. MAX es 'X', MIN es 'O'. 3. La evaluación debe considerar que X siempre empieza. 4. Incluir profundidad en el cálculo de utilidad para priorizar victorias más rápidas. 5. Método para mostrar estadísticas: nodos evaluados, profundidad alcanzada. El algoritmo debe ser completamente recursivo y explorar todo el árbol de juego hasta terminal."
Resultado / cómo se usó	Se obtuvo la clase Minimax con todos los métodos necesarios. La implementación recursiva evaluaba todo el árbol de juego y retornaba la mejor jugada. El conteo de nodos permitió medir el costo computacional de la

	búsqueda exhaustiva.
Validación / ajuste del equipo	Se verificó que el algoritmo siempre devolviera la jugada óptima en cada posición. Se ajustó la función de utilidad para penalizar victorias tardías (restando la profundidad). Se probó con todas las posiciones posibles del juego para validar el comportamiento. Se añadió un límite de profundidad para casos donde el juego podría ser infinito.

N°	P03
Objetivo del prompt	Implementar Minimax con poda Alfa-Beta, con estadísticas detalladas de nodos evaluados y podados, y comparación con la versión sin poda.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	“Ahora implementa la poda Alfa-Beta para la clase MinMax para Tic-Tac-Toe, considera: 1. Clase MinimaxPoda que extienda o replique la estructura de Minimax. 2. Método mejorJugadaConPoda(Tablero tablero, char jugador). 3. Método recursivo minimaxPoda(Tablero tablero, int profundidad, char jugador, int alfa, int beta). 4. Contar: - Nodos evaluados totales. - Nodos podados (cuando se corta la rama). - Profundidad máxima alcanzada. 5. Método para comparar con Minimax sin poda desde la misma posición. 6. Mostrar: - Nodos sin poda vs con poda. - Porcentaje de reducción. - Tiempo de ejecución de cada uno. Incluir comentarios explicando el funcionamiento de la poda y por qué es correcta.”
Resultado / cómo se usó	Se obtuvo la clase MinimaxPoda completa. La implementación incluía el mantenimiento de alfa y beta durante la recursión, y el conteo preciso de nodos podados. El método de comparación permitía ver la mejora de rendimiento en tiempo real.
Validación / ajuste del equipo	Se verificó que ambos algoritmos devolvieran la misma jugada para todas las posiciones de prueba. Se validó el conteo de nodos podados comparando con la teoría. Se ajustó el orden de exploración de movimientos para maximizar la poda. Se probó en todas las posiciones del juego para asegurar la consistencia.

N°	P04
Objetivo del prompt	Implementar las clases de jugadores (humano y máquina) y optimizar el ordenamiento de movimientos para mejorar la eficiencia de la poda Alfa-Beta.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para el Tic-Tac-Toe, necesito que apliques algunos cambios y estructuras lo siguiente: 1. Interfaz Jugador con método obtenerMovimiento(Tablero tablero). 2. Clase JugadorHumano que: - Solicite entrada por teclado (fila y columna). - Valide que la entrada sea correcta (0-2) y celda vacía. 3. Clase JugadorMaquina que: - Use Minimax con poda Alfa-Beta. - Permita configurar el algoritmo (con/sin poda). 4. Ordenamiento de movimientos (opcional): - Evaluar movimientos con heurística simple. - Ordenar movimientos para mejorar eficiencia de poda. 5. Método para visualizar el tablero después de cada movimiento. El ordenamiento de movimientos debe probarse y registrar su impacto en la poda."
Resultado / cómo se usó	Se obtuvieron clases de jugadores completamente funcionales. El ordenamiento de movimientos priorizaba el centro, luego esquinas, luego bordes, mejorando significativamente la eficiencia de la poda. La visualización del tablero después de cada movimiento mejoraba la experiencia de juego.
Validación / ajuste del equipo	Se probó el ordenamiento de movimientos en múltiples posiciones y se midió el impacto en la poda. Se ajustó la interfaz de JugadorHumano para manejar entradas inválidas de manera amigable.

N°	P05
Objetivo del prompt	Implementar los diferentes modos de juego (Humano vs Máquina, Máquina vs Máquina) con selección de algoritmo y configuración de profundidad.
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Crea una clase para gestionar las partidas de Tic-Tac-Toe que incuya: 1. Clase Juego que gestione:

	- El flujo del juego (turnos, validación de movimientos). - Verificación de estado terminal después de cada movimiento. - Anuncio del ganador o empate. 2. Modos de juego: - Humano (O) vs Máquina (X) - el humano elige su ficha. - Máquina (X) vs Máquina (O) - con selección de algoritmo para cada una. 3. Configuración de profundidad máxima para la máquina (fácil=1, medio=3, difícil=5, experto=9). 4. Menú principal que permita: - Seleccionar modo de juego. - Configurar profundidad. - Elegir algoritmo con/sin poda. - Iniciar partida. 5. Estadísticas acumuladas de la partida. El juego debe ser interactivo y fácil de usar."
Resultado / cómo se usó	Se obtuvo una clase Juego completa que gestiona todos los modos y configuraciones. El menú principal era claro y permitía navegar fácilmente entre opciones. Las estadísticas acumuladas mostraban el rendimiento de los algoritmos durante la partida.
Validación / ajuste del equipo	Se probaron todos los modos de juego con diferentes configuraciones de profundidad. Se verificó que la máquina nunca pierde (siempre empata o gana) cuando se usa profundidad total. Se detectó un error en el que la máquina siempre escoge el centro al inicio de la partida.

N°	P06
Objetivo del prompt	Realizar un análisis comparativo exhaustivo ejecutando múltiples partidas, generar tablas y gráficos de rendimiento, añadir la funcionalidad de decidir quien abre el juego y corregir que el primer movimiento de la máquina sea siempre el centro
Herramienta / modelo de IA	Claude 3.5 Sonnet
Texto del prompt	"Para finalizar el proyecto de Tic-Tac-Toe, realiza algunos ajustes y añade lo siguiente: 1. Clase AnalisisRendimiento que: - Ejecute 100 partidas máquina vs máquina (50 con X, 50 con O). - Compare Minimax sin poda vs con poda. - Mida: nodos promedio, tiempo promedio, profundidad promedio. 2. Tablas comparativas para profundidades 3, 5, 7 y 9. 3. Registro de: - Nodos evaluados vs profundidad.

	- Tiempo de ejecución vs profundidad. - Porcentaje de reducción con poda. 4. Añade la opción de decidir quien empieza la partida en el modo Humano (O) vs Maquina (X). 5. En el modo Humano (O) vs Maquina (X), la maquina siempre empieza por el centro, debido a que en el método de <code>findeBestMoves</code> de la clase <code>MinMax</code> tiene como prioridad principal el centro evitando movimientos en la esquinas, ajusta el algoritmo para que el primer movimiento pueda variar de la posición (1,1)
Resultado / cómo se usó	Se obtuvo un análisis completo con 100 partidas por configuración. Los datos generados permitieron crear tablas y gráficos detallados. Se añadieron las funciones requeridas y se corrigieron las clases mencionadas
Validación / ajuste del equipo	Se ejecutó el análisis en diferentes máquinas para validar la consistencia de los resultados. Se verificó que los porcentajes de reducción de nodos estuvieran dentro de los rangos esperados (50-75% aproximadamente).

9. Referencias

- Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 293-326. <https://kodu.ut.ee/~ahto/eio/2011.07.11/ab.pdf>
- Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. https://api.pageplace.de/preview/DT0400.9781292401171_A41586057/preview-9781292401171_A41586057.pdf